Unit-IV

Concurrency:

- Introduction
- Introduction to Subprogram level Concurrency
- Semaphores
- Monitors
- Message Passing
- Java threads
- Concurrency in functional languages
- Statement level concurrency.

Exception Handling and Event Handling:

→ Introduction
→ Exception handling in Ada
C++
Java
→ Introduction to event handling
→ Event handling with Java
C#

*Concurrency:

→ It refers to the ability of a program to execute multiple tasks or processes simultaneously.
→ Concurrency is when a program can do multiple things at the same time
- or switch b/w tasks quickly.
→ It improves efficiency and responsiveness.
→ Multiple processes are being executed during a period of time.

→ Concurrency can occurs at 4 levels:

1. Instruction level: Executing two or more machine instructions simultaneously.
2. Statement level: Executing two or more statements simultaneously.
3. Unit level: Executing two or more subprogram units simultaneously.
4. Program level: Executing two or more programs simultaneously.

→ Types of concurrency are:

1. Physical Concurrency
2. Logical concurrency.

* Subprogram Level Concurrency → means running multiple small tasks at the same time

* Same points of concurrency.

→ It allows the division of tasks into smaller subprograms that can run concurrently.

→ Subprograms can run on separate threads or processes.

→ Subprograms are typically executed on separate threads.

→ threads are light weight processes.

→ Subprograms should ideally perform independent.

→ This makes programs faster and more efficient.

→ It saves time.

→ Makes programs faster and responsive.

→ Multiple sub-tasks can execute at the same time.

→ Interaction among the processes take two forms:

* **Communication:** It involves exchange of data b/w Processes through messages or shared variables.

* **Synchronization:**

→ It involves synchronization of one task with another.

→ Generally thread execution is involved in synchronization.

→ By using appropriate concurrency models, the proper coordination of communication & synchronization b/w tasks can be achieved.

→ The concurrent models tell/dictate when & how the communication b/w the tasks should occur.

→ Concurrent programming can be done with the help of threads.

→ Thread is a lightweight process.

→ Each thread while running, executes some task.

* Semaphores → a technique to manage concurrent processes

→ It is an integer variable used to synchronize the progress of interacting processes.

→ Inorder to provide Synchronization has been developed.

→ Semaphore can be used to implement cooperation and competition synchronization.

→ the Semaphore works using two operations wait (P()) and signal V()

→ P() means try to decrease

→ V() means try to increase.

Working of Semaphore

1. Semaphore may be initialized to non negative value.
2. the wait operation decrements the semaphore value by one.
   - If value becomes negative, then the process executing wait() operation is blocked.
3. The signal operation increments the semaphore value by one.
   - If the value is not positive, then the process blocked by wait operation is unblocked.

Advantages

→ It allows only one process into critical section.

→ So, it is efficient method of Synchronization.

→ there is no wastage of resources.

→ Semaphores are machine independent.

Disadv

→ Code must be correctly implemented.

→ there are chances that low priority processes may access before high priority processes of critical section.

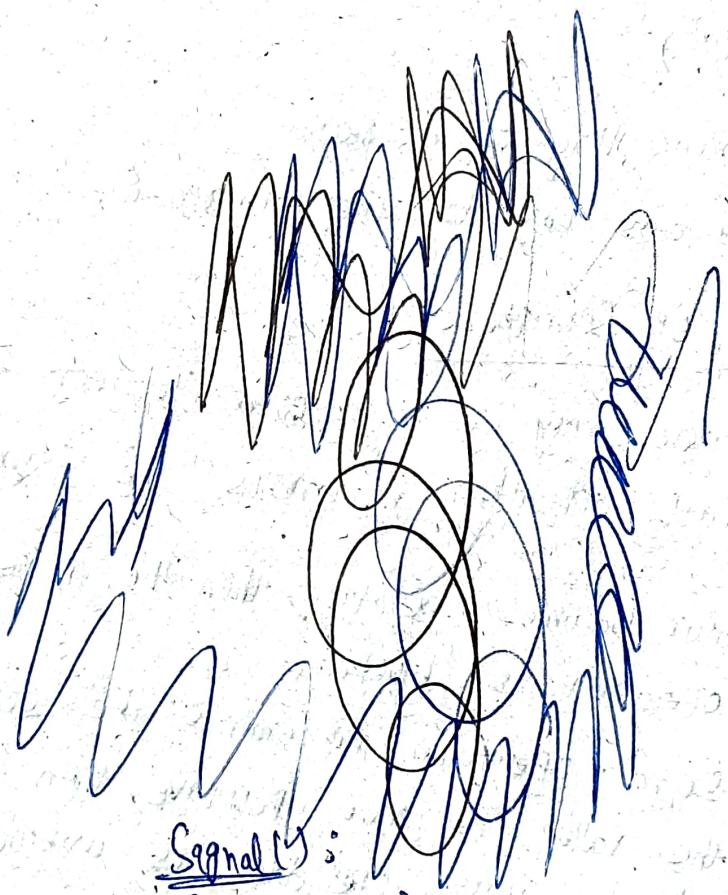
→ Semaphore is an integer variable.

→ It is used to manage concurrent processes.
  -by using a simple integer value (Semaphore)

→ It is a variable which is non-negative

→ It is used to solve the :- Critical Section problem

→ And to achieve Synchronization



Wait():

```
P(Semaphore S)
{
  while (S<=0)
  ;
  S--;
}
```

Signal():

```
V(Semaphore S)
{
  S++;
}
```

→ Semaphores are of two types: 1. Binary Semaphore
2. Counting Semaphore

1. Binary Semaphore: ↝ known as mutex locks

→ A Binary Semaphore is a Semaphore whose integer value ranges b/w 0 & 1.

→ It is used when there is only one shared resource.

→ It exists in two states Take & Give.

2. Counting Semaphore: ↝ Its value

→ To handle more than one shared resource of same type, counting Semaphore is used.

→ It is initialized with the count (N)

→ It will allocate resource

* ~~Monitors~~

→ Semaphore as an integer variable.

* Monitors → It is a synchronization tool,

→ It is used to implement process synchronization.
→ It is an alternative to semaphore.
→ Monitors are provided by programming languages
→ Java supports Monitors.
→ C doesn't supports monitor.
→ At a time only one process will be active in monitor.
* Same as Semaphore

* Message Passing

→ It is a mechanism of synchronizing & communicating among multiple processes.
→ The aim is to send & receive messages from one process to another process one object to another object.
→ It mainly consists of two operations:
- Send
- Receive

→ In this mechanism the sender & receiver must know each other.
→ there are two approaches of message passing :
1. Synchronous message passing
2. Asynchronous message passing.

1. Synchronous Message passing

→ In this method, the sender and receiver should wait for each other to transfer the message.
→ Sender will not continue its task until the message is received by the receiver.

Ex: Making a phone call.

→ We wait until the other person (receiver) accepts the call.

2. Asynchronous Message Passing

→ In this mechanism, the sender and receiver will not wait for each other.

→ Sender will send the message to receiver ~~with~~ without bothering whether the receiver is ready or not.

→ After sending the message, sender will continue with some other task.

Ex: Letter posting.

→ Sender posts letter in the post box.

→ Once the sender posts the letter, he will continue with other tasks.

* Concurrency in Functional Languages

→ Many functional Programming languages support concurrency,

1. Haskell
2. ML (Meta language)
3. Multilisp

→ Characteristics: Immutable data
Recursion

Benefits:

→ Improved Responsiveness

→ Simplified code

→ Improves System Performance.

Statement level concurrency

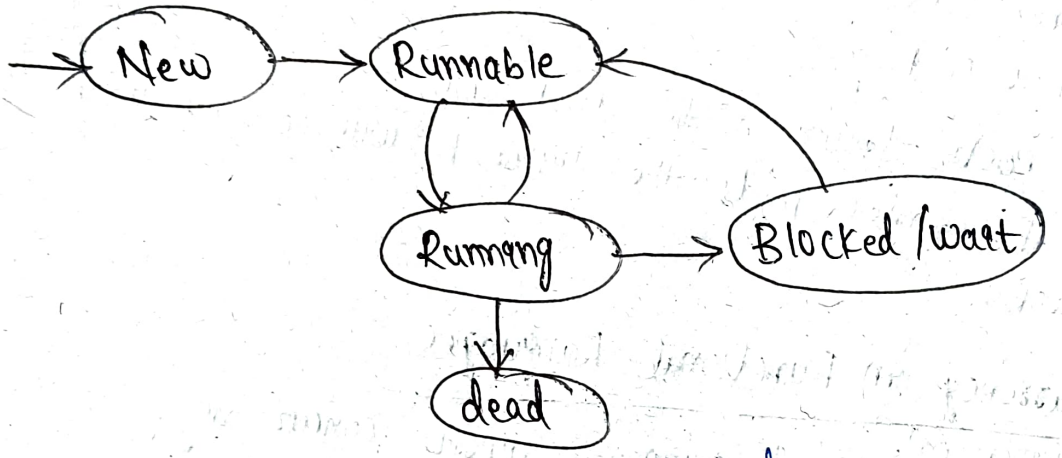
→ If two or more statements execute simultaneously.
→ we take multiple processor, to execute multiple tasks at a time.
→ It manages communication among processors.
→ A language which supports statement level concurrency is high-Performance fortran (HPF)
→ HPF is a collection of extensions to Fortran 90.
→ HPF defines some of the fundamental statements
- Such as: – no. of processors
– distribution of data over the memories
– alignment of data
→ ! – begin line of command.
→ Data distribution → Block
→ cyclic (single element)
→ Array

Ex: ! HPF$ PROCESSORS PROCS(n)
→ It specifies the no. of processors used

Java Threads

→ Thread is a light weight process.
→ Symbol of a thread: { { {
T1 T2 T3
→ Thread is a class.
→ A process contains multiple threads.
→ Threads are used to perform multiple tasks at a time.

Lifecycle of thread



Methods

- start()
- run()
- sleep()
- getPriority()
- setPriority()
- yield()
- resume()
- getId()
- isAlive()
- interrupt()
- suspend()

→ Creation of thread

```
Thread t=new Thread()
```

Ex:

```
class ThreadExample extends Thread
{
  public void run()
  {
    System.out.println("Thread is running");
  }
  Public Static void main(String args[])
  {
    ThreadExample t=new ThreadExample();
    t.start();
  }
```

Statement level concurrency

→ If two or more statements execute simultaneously.
→ We take multiprocessor, to execute multiple tasks at a time.
→ It manages communication among processors.
→ A language which supports statement level concurrency is high-Performance fortran (HPF)
→ HPF is a collection of extensions to Fortran 90.
→ HPF defines some of the fundamental statements
- Such as : – no. of processors
– distribution of data over the memories.
– alignment of data
→ ! – begin line of command.
→ Data distribution → Block
→ cyclic (single element)
→ Array

Ex: !HPF$ PROCESSORS PROCS(n)
→ It specifies the no. of processors used.

3. Exception handling and Multithreading

→ Generally, there are two major type error during writing a program:

① Compile time error → easily can be detected by compiler like Syntax error

② Runtime errors → called as Exceptions

these are some programming languages, they can't handle runtime errors

→ **Runtime error** (Def): the errors which occured during the execution of a program.

→ Java have a good mechanism to handle the runtime errors

Exception is unexpected/unwanted/abnormal situation that occured at runtime

→ An exception is a abnormal condition which arises during the execution of a program.

(or)

→ It is a runtime error that occurs during the execution of a program

How Java handles the exception

→ Java is having a very good mechanism in which it can easily handle runtime errors

→ Programming languages like C, doesn't have facility to handle these errors (exceptions.

→ Java handle runtime errors by providing 5 keywords:

try, catch, throw, throws, finally

Uses

1. try → used to monitor the Exception

→ If a particular part of a program is having a runtime error, in that circumstance, you need to put the code in try block.

2. Catch → used to handle the Exception

* If the code contain any runtime error, then that information will be pass to Catch block, Catch block catch the information & it will handle that Exception

try ↓ catch

3. throw → used to manually throw an Exception.

⇒ Whenever a program doesn't contain runtime error / Exception eventhough if we want to generate (create) an Exception at that time we'll make use of throw keyword

4. throws → Used by Methods to throw an Exception. (or)

* If a Particular method there is a chance in a method there is an Exception / If a Particular method want to throw an exception, then beside that method we need to make use of throws

→ It is ~~Specific~~ Specifically designed for methods

5. finally →

→ this block will be executed eventhough Exception has occurred or not.

→ No matter Exception has occured in our program or not, the code inside the finally block will be executed

Ex: If any accident happened when car is going in highway, no matter what other components will work or not but the airbags / the program of the airbags will be executed compulsory

Program:

```
class A
{
  public static void main (String[] args)
  {
    try
    {
      int a=10,b=0,c
      System.out.println (10/0);
    }
    catch (Exception e)
    {
      System.out.Println ("Error, can't devide by zero");
    }
  }
}
```

very imp

Ada Exception handling

→ the frame of an exception handler in Ada is either subprogram, a package body, a task or a block.

→ Exception handler have the following general form

exception
when [exception_name]
⇒ statement
- - - -
when ... [exception_name] ... ⇒ [statement]
- - -
when [other ⇒ statement_sequence]

→ Handlers are place at the end of the block or unit.

Binding Exception to handler :

→ If exception raise, but does not have handler

1. Procedures – Propagate goes to calling function
2. Block – Propagate to scope in which it occurs
3. Package body : Propagate to the declaration part.
4. Task – No propagate, no handler, execute it, mark it complete

face.

* Exception handling in C++

Exception:

→ It is an event which occurs during the execution of program
- that disturbs the normal flow of program instruction.

Ex: the number divide by zero

⇒ compiler successfully, but an exception (or) run-time error will occur,
- due to which our application will be crash.

→ Inorder to avoid this, we will introduce Exceptional handling in our code.

→ the process of converting system error message into user-friendly error message is known as "Exceptional handling".

Exception handling Mechanism

1. Find problem (Hit the exception)
2. ~~Send~~ Inform about its occurence (throw the exception)
3. Receive error information (catch the exception)
4. Take proper action (handle the exception)

→ C++ error handling keyword : Try, catch, throw
(catch → we find error; detect error)

Syntax:

```
try {
// code (that is exepected to raise exception)
}
catch (formal Parameter){
  // ** handler body
}
.
.
catch (formal parameter) {
// ** handler body
}
```

Event handler

→ An event is a notification that something specific has occured such as a mouse click on a graphical button.

→ the event handler is a segment of code that is executed in response to an event.

Event model in Java:

→ The event model is based on the event source & event listener.

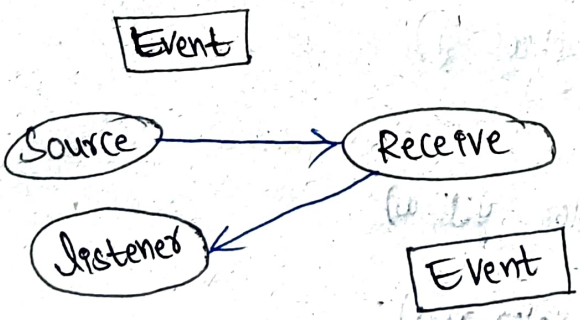
→ Event listener is any object that receive a message/event.

→ Event source is any object creates a message/event.

→ Event delegation model is based on event classes, event listeners, event object.

→ three participants in event delegate model in Java

1) Event source - the class which broadcast the event.
2) Event listeners - the class which receive notification of event.
3) Event object - the class which describe the event.



Java Event model

→ One class of event is item event, which is associated with the event of clicking, check box, radio button.